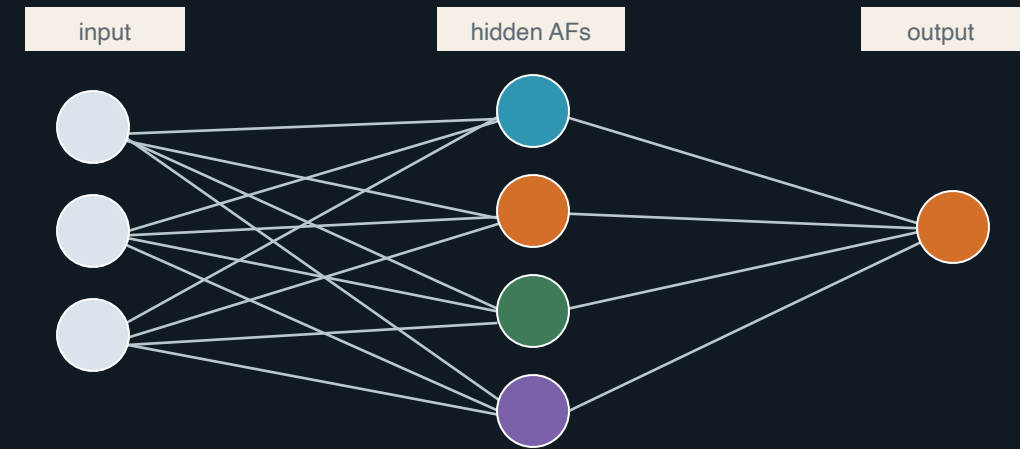


Simulated Annealing for Activation Function Layouts

A discrete search method for assigning activation functions inside a basic neural network.



state = activation layout

the search state is not a weight matrix

goal = low validation loss

after short candidate training

Simulated Annealing is a search heuristic for large discrete spaces.

Analogy

A hot material can move freely. As it cools, it settles into a stable low-energy structure.

Optimization view

The algorithm accepts good moves and sometimes accepts worse moves to escape local minima.

Why useful here

Activation layouts are discrete. There is no gradient telling us which activation to choose next.

search: current state → neighbor state → accept or reject

repeat while temperature and budget allow exploration

The optimization target is the activation layout, not the weights.

Gradient descent

Trains weights and biases for one fixed layout.

Continuous parameters:

$w_1, w_2, \dots, b$

Simulated annealing

Searches over assignments of activation functions.

Discrete choices:

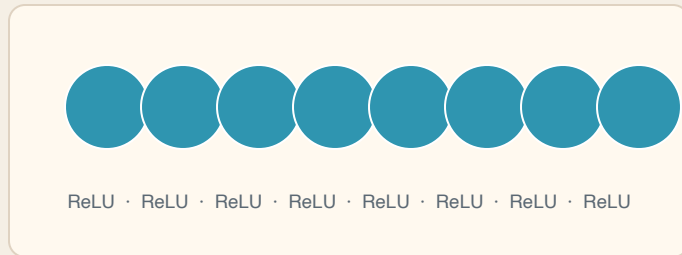
ReLU, Sigmoid, tanh, ...

layout $L = [f_1, f_2, \dots, f_n]$ with $f_i \in \text{AF-set}$

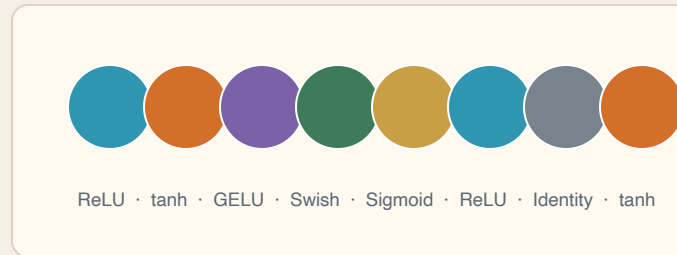
weights are trained inside each candidate evaluation

A state is a concrete activation assignment for the hidden neurons.

homogeneous layout



mixed layout

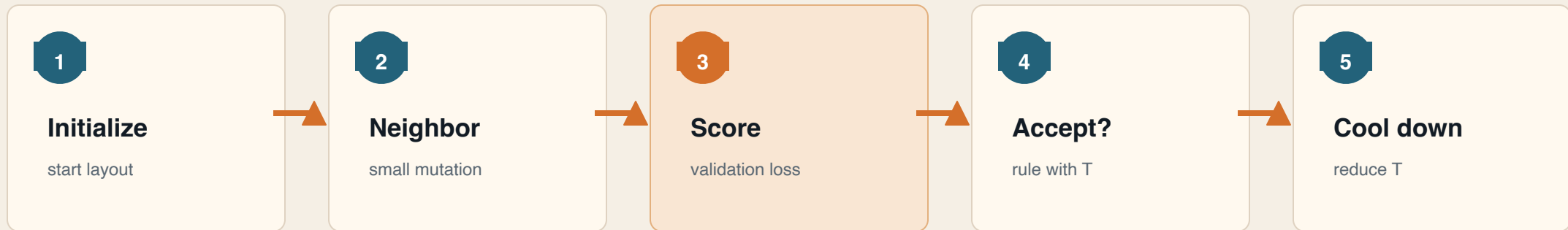


layer-filled layout



- The network topology stays fixed.
- Only the activation function distribution changes.
- Each state can be trained and scored under the same data split.

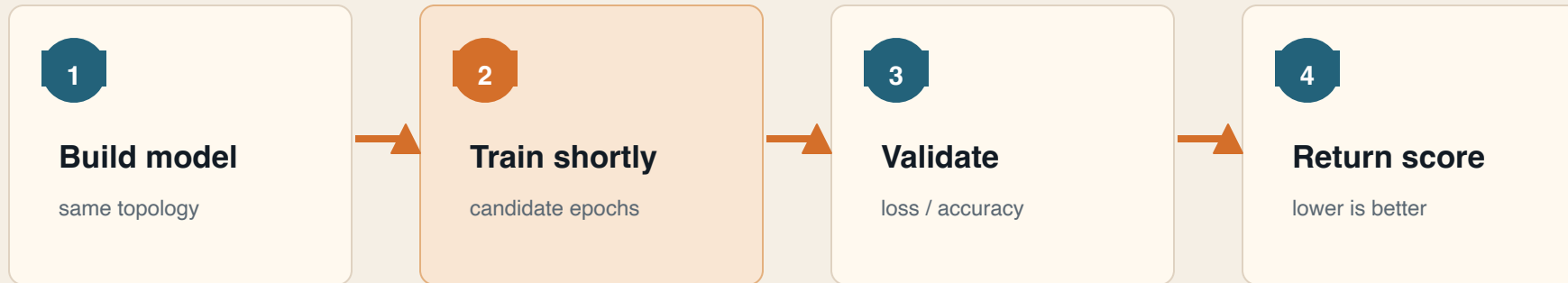
The SA loop repeatedly proposes, scores, and decides on layout changes.



repeat until max_steps or temperature stop condition

best layout is tracked separately from the current layout

Candidate scoring converts a layout into one comparable number.



Important distinction

Candidate training is a cheap estimate used during search. It is not the final performance report.

Validation rule

Validation data guides search. Test data is reserved for final evaluation only.

The acceptance rule allows controlled uphill moves.

Improved candidate

$\Delta = \text{candidate_score} - \text{current_score}$

if $\Delta \leq 0$: accept

Worse candidate

if $\Delta > 0$: accept with probability

Best state

Best layout is stored separately even if the current state later moves away.

$$p = \exp(-\Delta / T)$$

large T makes worse moves more likely; small T makes them rare

Temperature controls exploration at the beginning and convergence later.

High temperature

Worse candidates may still be accepted.

Purpose: escape local minima and explore different regions.



Low temperature

The algorithm becomes more conservative.

Purpose: stabilize around promising layouts.



$$T_{k+1} = \alpha \cdot T_k \quad \text{with} \quad 0 < \alpha < 1$$

geometric cooling is simple, deterministic, and easy to compare

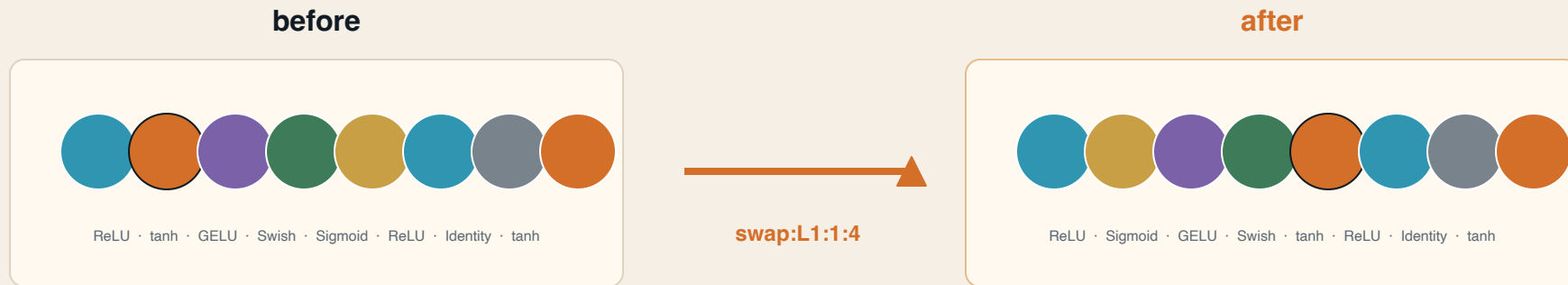
``set_neuron`` changes exactly one activation choice.



Use case

Fine-grained local search. Good when the current layout is mostly plausible and only individual neurons should change.

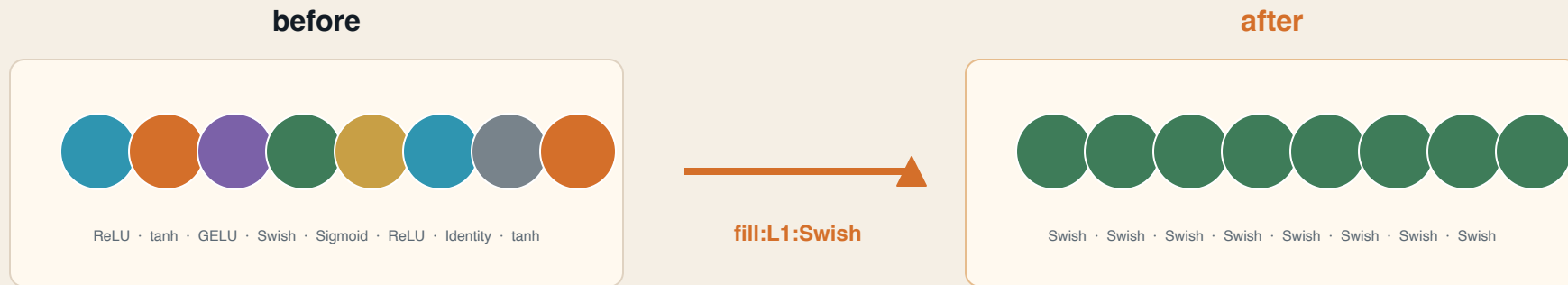
``swap_neurons`` keeps the activation multiset but changes positions.



Use case

Tests whether placement matters while preserving the number of each activation function.

``fill_layer`` creates a larger structural jump.



Use case

Coarse search step. Useful for leaving plateaus, but it can also destroy useful local structure.

The most important hyperparameters define the search budget and behavior.

max_steps

How many candidate moves can be tested.

candidate_epochs

How much training each candidate receives before scoring.

start_temperature

How exploratory the early search is.

cooling α

How fast exploration is reduced.

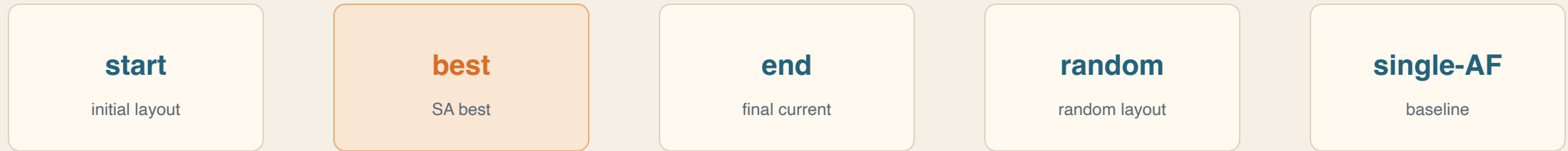
neighborhood set

Which operations are allowed: set, swap, fill, or combinations.

seeds

Multiple seeds are required because both training and search are stochastic.

Final evaluation must retrain selected layouts under identical conditions.



same split · same topology · same epochs · same seeds

search score and final test performance are reported separately

During a run, these signals explain what the search is doing.

current layout

The state the algorithm is currently moving from.

candidate layout

The neighbor being tested in this step.

best layout

The best validation score seen so far.

temperature

The current exploration level.

- Falling temperature means fewer accepted worse moves.
- Acceptance rate shows whether the search still explores.
- Validation loss compares candidates; test accuracy is only final evidence.

The method is useful, but it does not guarantee a global optimum.

What SA provides

A structured, budget-controlled search through a discrete activation layout space.

It can escape local minima by accepting some worse candidates.

What still needs validation

Hyperparameters, neighborhood choice, and training budget must be compared over multiple seeds.

Final claims require fair retraining.

SA searches activation layouts; gradient descent trains the weights.

this separation is the central methodological point